这大概是我读过最硬核的一次 Linux 内核重构文章了

腾讯技术工程 2026年4月12日 02:51 广东 47人



作者：TencentOS

背景

Linux 内核的 Swap 子系统自诞生以来，数十年间代码复杂度持续累积，逐渐成为内存管理子系统中公认的复杂地带。然而，来自腾讯的内核工程师 Kairui Song 以一系列系统性的重构方案，彻底扭转这一局面。

自2025年 Linux 存储、文件系统、内存管理与BPF峰会上首次亮相以来，腾讯服务器操作系统 TencentOS 内核研发工程师 KairuiSong 主导的Swap现代化工程，历经了多个阶段的演进：swap table 的引入（已合并进 Linux 6.18）从根本上替代了沿用多年的 XArray 结构，带来5%~20%的性能提升；swap map 的移除（面向 Linux 7.1）进一步统一了 Swap 元数据管理，节省约 30% 的元数据内存开销；而将整个 Swap 子系统重新构建于更为清晰、高效的数据结构之上的宏大目标，正逐步从设想变为现实。

这项持续了18个月的工作在 Linux 内核社区引发了广泛关注。LWN.net 知名作者、Linux 内核文档核心维护者 Jonathan Corbet 专门撰写了连续三篇深度分析文章（"Modernizing swapping" 系列），逐篇解析 Kairui 及其协作者（Chris Li 等）所做工作的技术细节与及其深远影响——这在 LWN 的报道传统中，是对一项内核贡献极高规格的认可。

本文对 Corbet 的三篇文章进行了系统翻译与整理，以期将这项重要的内核技术进展完整呈现。（原文链接详见文末附录）

一、引入swap table

内核的 Swap 子系统是一个复杂且常被忽视的庞然大物，但它也是内存管理子系统的关键组成部分，对整个系统的性能有重大影响。在 2025 年的LSF/MM/BPF（Linux 存储、文件系统、内存管理和 BPF）峰会上，Kairui Song 提出了一个简化并优化内核交换代码的方案[1]。该工作的第一部分[2]（由 Chris Li协助）成功进入6.18 版本内核，开启了整个 Swap 子系统现代化的序章。

1.1 虚拟内存与匿名页换出

在虚拟内存系统中，内存不足时必须通过页面回收来应对，在必要时需要将内存内容写入持久化后备存储。对于文件页面来说（也就是 page cache），文件本身即为后备存储。但匿名页（anonymous pages）——也就是用于存放进程的堆、栈及各类数据结构的数据区，天然不存在这样的后备存储。这正是 Swap 子系统存在的意义：当匿名页所占用的物理内存需要被回收时，Swap 子系统为其提供写出目标。换出（swapping out）将不活跃（或访问频率低）的页面推送到持久化存储介质，从而释放内存，腾出空间当前活跃使用的工作负载。

1.2 Swap子系统快速入门

然而完整地描述内核的 Swap 子系统确实会很冗长；其中有着巨量的随着时间逐渐积累起来的复杂问题。以下是 Linux 内核 Swap 子系统在 6.17 版本下的简述。

Swap 子系统支持使用一个或多个 swap 文件，可以是存储设备上的独立分区，也可以是文件系统内的普通文件。内核中，可用的 swap 文件由 struct swap_info_struct[3] 描述，通常以简单整数索引来引用（内核中每个swap_info_struct拥有一个独立的 type 作为标识符——其实这也是个历史遗留问题，这里的 type 更应该叫作id或index）。每个 swap 文件内部被划分为页面大小的槽（page-sized slots），所有 Swap 文件上的任意槽位都可以使用 swp_entry_t[4] 类型来索引到。

```
typedef struct {
    unsigned long val;
} swp_entry_t;
```

这个 longval 被分为两个字段：高六位是 swap 文件的索引号（也就是上文提到的type），其余部分是文件内的槽位编号（在 swap 文件内部的偏移量）。内核提供了一组简单的函数[5] 用于创建 swap entry 并获取相关信息。

需要注意，上述为架构无关的通用格式。各体系结构还有其架构相关的变体，用于避免与页表项（PTE）格式产生冲突。好奇的读者可参阅x86_64 宏[6]，两种形式之间可以进行转换。在 Swap 子系统内部，统一使用架构无关版本。

1.2.1 换出流程简述

当内存管理子系统决定回收一个匿名页时，它就会选择一个 swap 槽位，将该页内容写入该槽位，然后将对应的 swap entry（以架构相关格式）存入页表项（PTE）。swap entry 存储在PTE中时，其格式是不包含PTE所定义的"present" 位的，因此下次访问该页时将触发缺页异常（page fault）。内核识别到 swap entry 后，分配新的物理页，从 swap 文件读取内容，并相应更新 PTE。

现实中的流程远比上述复杂。例如，将页面写入 swap 文件需要时间，在写完之前该页无法被真正回收。因此，回收决策做出时，该页首先被置入 swap cache（与文件映射页使用的 page cache 高度类似）。页面位于 swap cache，也就意味着已为其分配了一个 swap entry。该页本身仍驻留于内存的这段时间便会与该 swap entry成为绑定关系，用于处理各种同步和反向查询等操作。例如若写入过程中发生缺页异常，该页可被迅速重新激活（reactivated）。内核在等待页面完成回写后，才可以真正将其释放。同时也有类似于ZRAM这样的快速设备，其上的回写极其迅速因此这里的等待操作会更简洁。

1.2.2 address_space 与 XArray

如同内核文件系统需要维护 page cache 中每个页面的状态，swap 子系统在处理分配 swap 槽位和IO以外，也需要追踪 swap cache 中每个页面的状态。在 6.18 之前的内核中，swap 子系统维护了一个名为swapper_spaces[7] 的结构，复用文件系统常见的address_space[8] 结构从而维护了Swap地址空间（此处文件的内部偏移和swap 文件的槽位偏移非常接近），从而直接维持了 swap 后备存储与页面之间的映射关系，并提供一组在 RAM 与后备存储间移动页面的操作集。这里复用的好处之一，就是使很多 page cache 相关代码也能直接作用于 swap cache。

因此长期以来，swap cache 与 page cache 共用 XArray 数据结构作为其核心数据结构。XArray 可以存储每个槽位的当前状态，每个槽位状态可以是：

●空，Xarrayentry为NULL

●槽位已分配，且页面仍驻留于 RAM：XArray entry 为指向该页（更准确地说，是包含该页的 folio）的指针

●槽位已分配，但页面已释放，数据只存在于Swap文件中。在这种情况下，Xarray entry包含“影子”信息，供内存管理系统用于检测换出后快速错误进入的页面。（有关此机制的概述，请参阅这篇2012 年的文章[9]）

此外，每个swap文件并非只有一个 address_space 结构和XArray[10]。相反，swap文件会被划分为 64MB 的块，从而将数据分散到多个 XArray 上，减少Xarray锁竞争提升大型系统的可扩展性。

1.2.3 Swap Cluster

Swap 子系统还引入了另一层管理机制：每个 swap文件内部被划分为若干 swap cluster（由 struct swap_cluster_info[11]描述，通常大小为 2MB）。swap cluster 使 swap 文件的管理具有更好的可扩展性：系统中每个 CPU 维护一个本地cluster，相关 swap entry 可完全在 CPU 本地管理，只有在 cluster 级别的分配或释放时才需跨 CPU 访问。更早时Swap子系统还会给每个CPU分配一个更细粒度的槽位缓存，这一设计已经随着swap cluster的进一步优化和简化而移除。

Swap cluster 减少了对全局 swap map 的扫描，但对特定槽位状态的读写仍需访问对应的 XArray。

1.3 The Swap Table

有了上述背景，便可深入理解 6.18 的改动。

注意到，Swap 子系统处理 swap entry时就已经能得知其所属的swap cluster。若将所有状态信息（包括Swap cache）随 cluster 一并存储，便可消除 XArray，代之以使用更简单和高性能的C数组。将整个Swap cache的管理局部化，简化，从而提升可扩展性。

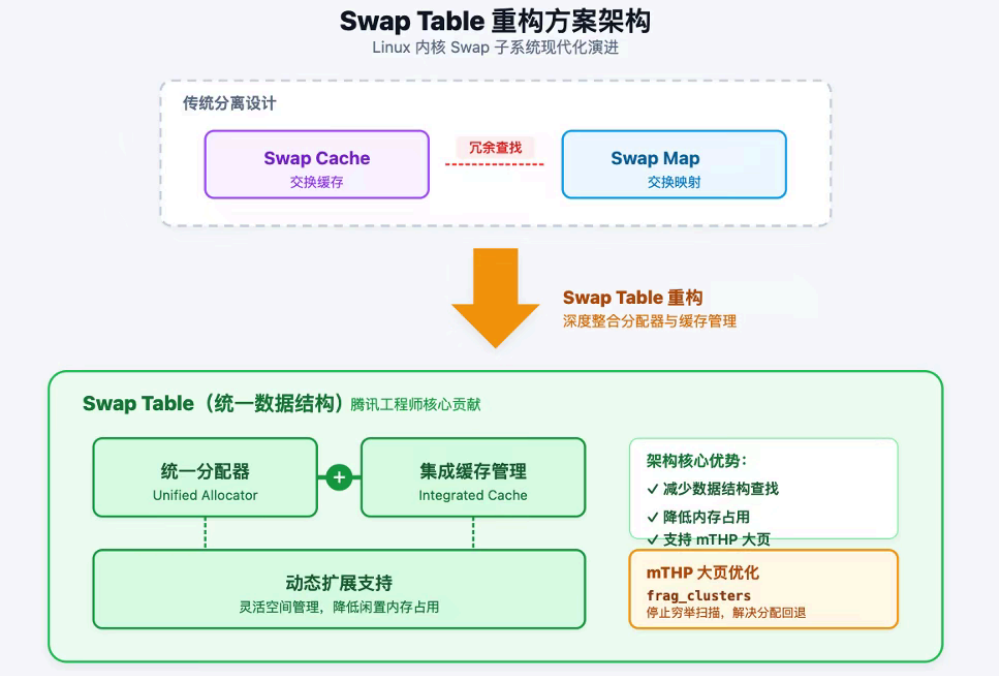
因此，SwapTable第一阶段补丁集增强了 swap_cluster_info 结构；6.17 之后的版本结构[12]新增了一个数组指针：

```
1  atomic_long_t __rcu *table;
```

新的table 数组被设计为在多数体系结构上恰好占用一个物理页，并采用动态分配，只有一个cluster被真正使用时才会分配，以此在 swap 文件未满时可减少 swap 子系统的内存占用。数组中的每个条目均为一个swap table entry，其直接描述 swap文件中某个槽位的状态。Swap 代码进行了大量重构以使用这一全新的核心数据结构。大多数内核内部 API 只需极少甚至无需改动便可兼容。

如此以来，内核此前所采用两套独立的聚簇机制（address_space和swap cluster），现在统一为单一聚簇方案，进一步降低了锁争抢的同时，简化了代码和设计。原本每 64MB 一个的 address_space 结构体数组也已移除，XArray 不再需要，swap地址空间操作可由单一结构体 swap_space[13] 的单一结构提供。

根据 Kairui 的测评，第一阶段工作就可以**“在基准测试和工作负载测试中，吞吐量、RPS 或构建时间性能提升高达约 5-20%”**。这一性能提升主要源于消除 XArray 查找，以及通过在更小粒度上管理 swap 空间所带来的竞争减少。



二、删除Swap Map

Swap map 是当前持续推进的 swap 改进工作的第二个核心目标。乍一看，当前内核中的 swap map 是非常轻量和简单的数组，存储在 struct swap_info_struct 中：

```
unsigned char *swap_map;  /* vmalloc'ed array of usage counts */
```

该数组中，Swap 设备中的每个槽位对应一个字节。每个字节存储的值是指向该槽位的引用数量。不论分配给该槽位的页面是否驻留在 RAM 中，每一个指向该槽位的页表项（PTE）都会贡献一个引用计数。

当然，swap系统错综复杂，情况自然不会那么简单——Swap map里每个entry都预留一些特殊bit。其中对本文最重要的是第 6 位（0x40），称为 SWAP_HAS_CACHE，用于表示某个槽位所

对应的页面是否还在内存里。在某些时间窗口内，swap slot 可能已被分配，但尚未有页表项引用它，此时引用计数为零。SWAP_HAS_CACHE bit 用于区分这种状态与 slot 未被分配的状态。

这个标志还被用作位锁（bit lock）同步。内核常常需要并行地多次尝试换入同一页面（或进行操作）。在这种情况下，竞争线程会以SWAP_HAS_CACHE bit作为同步信号，成功设置这一bit的线程才能进行工作。这引发了不少问题：Swap 代码中有大量延迟重试循环（例如[14]）或其他 work around。

Swap map中还有一些其他特殊值；例如 0x3f：SWAP_MAP_BAD，表示底层存储槽不可用。更特殊的是0x3e：SWAP_MAP_MAX表示引用计数超过了swap map可承载范围。Swap map使用byte作为基础单位，因此只能支持最大为 62的引用计数。这带来了一个问题：当大量任务共享一个匿名页面时，引用计数很容易超过这个值。处理这种情况的方式则更是非常繁琐。

每次 swap slot 的引用计数递增时，都必须检查是否溢出。发生一溢出时，另一个预留标记位 0x80（COUNT_CONTINUED）将被置位。Swap map 中的计数将被清零，并分配一个新的物理页，为引用计数提供更高的 8 位。该新页通过关联 page 结构中的 LRU 链表头链接到原swap map页。如果溢出页中的条目再次发生溢出，就继续分配一个页面追加到链表中。

考虑低位仍旧保留在swap map中，大部分引用计数操作不需要走到额外页面里，这样的设计尚可接受。当引用计数较低时（这也是常见情况），这种设计提供了最小的内存开销和不错的性能。

2.1 Swap cache bypass 与 SWAP_HAS_CACHE

正如我们提到的，swap cache的用途之一是持有追踪槽位与页面之间的关系，这在换入换出IO过程中相当重要。如果对一个已换出的 folio 发生了缺页中断，就需要分配一个新 folio 并从 swap 文件中读取其内容。读操作往往需要一定时间，因此该 folio 会被加入 swap cache，直至读取完成才会从中移除，竞争的进程看到该页面便可直接复用和避免忙等待或重复IO。通常，swap子系统还会尝试预读（readahead），预读页面同样缓存于swap cache。

这一情况在 2018 年的 4.15 版本中有所改变：Swap文件的IO有时非常迅速，特别是对于ZRAM这样的内存压缩方案。在这类设备上，swap cache的额外开销和预读等行为非但不能提升性能，反而可能有害。

因此在 4.15 版本中，Minchan Kim 加入了Swap bypass特性。如果Swap 设备设置了 SWP_SYNCHRONOUS_IO 标志（表示设备速度极快，I/O 应同步执行），且 swap map 中特定 slot 的引用计数为 1，那么换入该 slot 中页面的请求将同步执行，不进行预读，且不加入 swap cache。这给 swap 子系统又增加了相当多的复杂性，多年来引发了各种 bug，但也带来了显著的性能提升。这种提升来自两个因素：避免了相对昂贵的swap cache操作开销，以及阻止了预读的使用。

而在7.0内核中，Kairui Song 的swap table第二阶段补丁系列的第一部则致力于完全移除这一 bypass 特性。Kairui Song第一阶段的工作——也就是swap table的引入，让swap cache操作速度大幅提升，以至于对于即使像ZRAM这样的设备，绕过 swap cache 也不再有实质价值。该系列后续又将 read ahead 的控制独立出来，并对快速设备基本上完全禁用了预读。让所有swap I/O 都通过 swap cache，不仅简化了代码，更减少了令人头疼的竞态条件。

新代码对于 SWP_SYNCHRONOUS_IO 设备，会在换入完成后立即将 folio 从 swap cache 中移除，以此释放 swap 数据所占用的内存，但完整使用swap cache作为同步原语，避免忙等待和优先级反转等问题。

移除 swap bypass 还带来一个有趣的副作用：在当前内核中，高阶folio（THP/mTHP）只能通过 bypass 路径换入，所以也只有在引用计数为 1 时才能完整换入。移除 bypass 特性后，无论引用计数为何值，都可以换入完整的folio。

这也更进一步简化了swap的操作原语，swap的所有管理和操作都被整合进一组定义明确的小函数中。这些函数全部基于 folio和swap cache，从而减少了 swap 子系统历史上以swap entry为中心的设计。借助这一设计，外加上使用cluster lock 与 folio lock 的组合来管理 swap cache，只需再迈一步，就可以用这两把锁来控制 swap map 的访问了。

一旦 swap cache 承担起管理并发的职责， SWAP_HAS_CACHE bit 就只剩最后一个用途：标记已分配但引用计数为零的 swap slot。在换出侧，通过在 slot 分配后立即将 folio 加入 swap cache，消除了这种状态的存在。在另一端，当页面从 swap cache中移除时，引用计数为零的 swap slot会被立即释放。至此，SWAP_HAS_CACHE 不再有存在的必要；系列末尾的这个补丁将其彻底移除。

2.2 移除swap map

上述工作截至本文撰写时已进入 7.0 版本主线。后续可能在7.1 之后的版本中，我们会完成 swap table 的第三阶段，彻底消灭 swap map。

在新的swap table 中，每个entry是一个64位（或32位，取决于架构）unsigned long。目前为止其中的内容与之前内核 XArray 数据结构中存储的高度相同——NULL（0）表示空 slot；对于驻留 folio，条目存储 folio 地址；对于已换出 folio，条目存储用于追踪哪些页面能被快速从 swap 换回的 shadow 信息。第三阶段改变了这张表的条目格式，以支持五种不同类型的条目：

●NULL（0）：仍表示空 slot。

●低一位置 1：这是一个已换出 folio 的 shadow entry，条目高位存储该条目的引用计数。可用于引用计数的 bit 数量因架构而异。

●低两位置10 ： 该条目对应一个驻留内存的 folio。与 shadow entry 类似，最高位存储引用计数；为了给引用计数腾出空间，存储的是底层页面的**页帧号（PFN）**而非其地址。

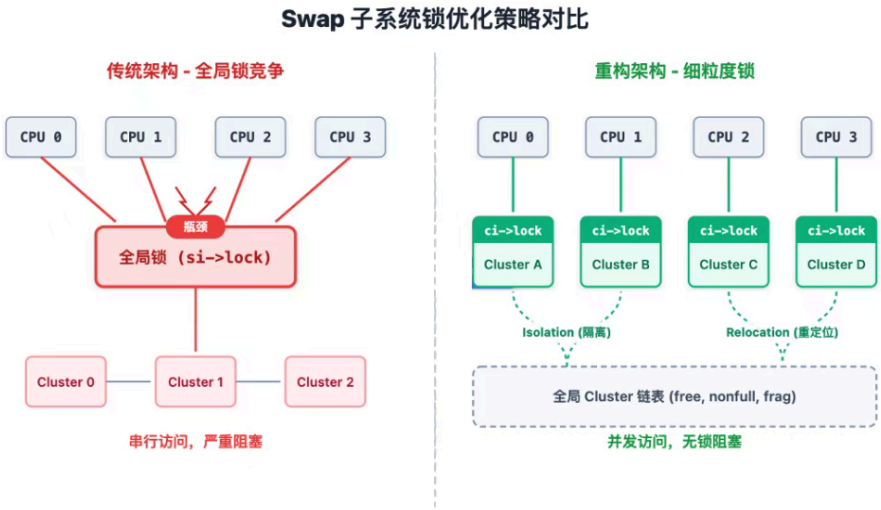
●低三位置100 ： pointer entry，当前系列中未使用。

●低四位置1000 ： 标记为坏 slot，不应使用。

这种设计将 swap map 最后剩余的职责——追踪引用计数——挤进了 swap table，从而使 swap map 得以彻底移除。长期以来内核一直将 swap map和 swap cache作为两个独立的数据结构分开维护。这里将这两者统一，显著减少了记录维护的开销，从整体上加快了 swap 系统的速度，以及相当可观的内存节省。Swap 子系统约 30% 的元数据开销得以消除，对于 1TB 的 swap 文件来说，可节省约 256MB 内存。

新格式仍旧只是用了有限数量的bit来引对大部分工作负载，而在处理大量共享内存的工作负载中可能出现的溢出上，也更加直观。如果引用计数溢出，将为整个 cluster 分配一个单独的 unsigned long 计数数组。

第三阶段目前已经进入了mm-unstable等待合并到主线，但工作也还没有结束：Kairui提到了后续阶段，将把 memory controller（内存控制器）对 swap 的限额管理也整合进 swap table。因此，和内核的其他部分一样，swap 子系统恐怕在很长时间内都会被认为尚未"完工"的状态。



三、 Swap Table 的完善与虚拟Swap Table

Swap 的故事并不仅限于此。在完成了整个系统现代化重整后，大量原本难以实现的特性现在也变得可行。在过去多年的讨论中，虚拟 Swap 这一概念被广泛提及。近期Meta以及TencentOS团队都分别提出了各自的解决方案，来引入虚拟 Swap 空间这一概念。

其中Meta的方案[15]是引入一个单一且必选的中间层作为虚拟 swap 空间（virtual swap space），解决了一系列与 swap 相关的问题。同时，腾讯的TencentOS团队也提出了全新的 Virtual GhostSwap方案[16]，其结合了Google的GhostSwap方案以及众多社区需求，以更优雅，更灵活的方式实现虚拟 Swap 空间。

3.1 背景与动机

对于虚拟Swap的需求和性能平衡问题，Kairui 始终认为虚拟层应该是可选的，而不是强制的，且性能与内存开销不能造成显著问题。在目前已有的 Swap 重构的基础上，借助 Swap Table 机制，Swap table 第四阶段的目标，正是从根本上解决上述问题。

3.2 进一步系统性重构：统一分配路径

这一阶段的核心工作并非针对某一具体 bug 的修补，而是对整个 anon/shmem swap 分配流程的全面重构。此前，匿名页和共享内存的换出路径在 swap cache 操作、slot 分配与 memcg 计费上均存在各自的逻辑分支，代码复杂且同步原语分散。本阶段将两条路径统一为基于 folio 的规范流程：swap cache 层负责确保 folio 对 swap slot 的独占所有权后再行计费，从根本上消除了压力场景下因竞态引发的内存抖动（thrashing），以及偶发的 VM_FAULT_OOM 泄漏问题。

3.3 移除 swap cgroup 静态数组

在此次重构中， mm/swap_cgroup.c ——一个独立维护 swap slot 与 cgroup 映射关系的静态数组模块——被完整移除（共 172 行代码）。其功能被直接整合进 swap table，由后者统一承担 memcg 信息的存储与检索职责。

这带来了显著的内存节省：以挂载一块 1TB 的 swap 设备为例，这一改动可节省约 512MB 的内存。

3.4 开创性工作：Dynamic Ghost Swapfile

第四阶段还附带了一组 RFC 性质的补丁，引入了 Dynamic Ghost Swapfile 的概念——这是 swap 子系统在架构层面的一次前瞻性探索，以动态的方式引入虚拟Swap。

其核心思路是：利用 Xarray 构建动态大小的虚拟 swap 文件，使 swap 空间的边界不再受制于预先分配的静态结构。单次 swapon /dev/ghostswap 即可获得近乎 PB 级虚拟 swap 空间，且对现有用户零影响。相比一个强制的中间层，动态虚拟swap在性能和可扩展性上都有显著优势。当然，社区在技术方案上的分歧解决通常都不会太顺风顺水，这可能也会是一项长期的工作。

3.5 社区协作与影响

本阶段工作也由Kairui Song 主导，并得到 Google 的 Chris Li 在 Ghost Swapfile 方向上的重要协作贡献。补丁发出后社区有着大量的讨论和Review，该RFC后续应该会继续在社区中迭代完善。

四、虚拟 swap 空间——两种方案的碰撞

目前两种不同的虚拟 Swap 实现方案仍在社区推进中。其中基于 SwapTable 的方案需要进一步的基础设施改造才能有望进一步推进。同时 Virtual Swapspace方案的推进也在持续进行。在 Virtual Swapspace 方案中，Pham也明确阐述了改方案想要解决的问题和设计思路。

4.1 Swap entries的痛点

正如我们提到的，swap entry用于标识 swap 设备上可用于存放一页数据的槽位（slot），包含设备索引和设备内的偏移量。当一个匿名页被换出到 swap 设备时，对应的 swap entry 将被写入所有指向该页的页表项（PTE）。借助该 entry，内核可在需要将页面重新 fault in 到 RAM 时，迅速定位已换出的页面。

这里的问题是，每个swap entry直接存入PTE，这导致PTE和swap 设备变成了绑定关系，这给系统管理员和系统设计者带来了一定的麻烦。

问题一：swap off 设备代价高昂

Swap off 会尝试移除一个 swap 设备：在设备移除之前，存储于其上的所有数据页必须先被 fault in 回 RAM，这一步无可避免。但还存在额外的问题——我们必须确保所有的PTE都不再指

向这一设备。为解决这一问题，内核必须在移除时扫描系统中全部匿名页的页表项，并将其更新为页面的新位置。这一过程相当耗时。

问题二：zswap 的困境

这种设计也给zswap子系统[17]的用户带来麻烦。zswap 的工作原理是：在换出流程中拦截页面，不将其写入磁盘，而是将其压缩后存回内存。这也就导致，zswap无法在没有真实物理设备时使用，且zswap可压缩内存的量完全限制于物理块设备的大小：页面首次进入 zswap 时，必须立即在后备设备上预分配一个槽位。这意味着，即使从不打算真正将页面写入磁盘，每次 zswap 使用都必须在后备设备上占用空间，造成大量存储空间浪费，甚至使 zswap 在没有可用后备存储的系统上无法使用。

4.2 虚拟swap空间

Meta的Pham提出的解决方案，正如计算机领域的常见的做法：增加一层中间层。具体而言，就是用一张独立于底层设备的统一空间用来分配swap entry，此时所分配的swap entry 是一个 swp_desc 结构体数组：

```
struct swp_desc {
    union {
        swp_slot_t          slot;
        struct zswap_entry * zswap_entry;
    };
    union {
        struct folio *      swap_cache;
        void *              shadow;
    };
    unsigned int                swap_count;
    unsigned short              memcgid:16;
    bool                        in_swapcache:1;
    enum swap_type              type:2;
};
```

第一个 union 告知系统已换出页面的位置：要么指向设备特定的 swap 槽位，要么指向 zswap cache 中的一个 entry，即虚拟 swap 槽位与真实位置之间的映射。

第二个 union 包含页面在 RAM 中的位置（更准确地说是其 folio），或内存管理子系统用于跟踪页面 fault-in 速度的 shadow 信息。

swap_count 字段记录有多少个页表项引用了此 swap 槽位； in_swapcache 在页面被分配至该槽位时置位； memcgid 记录管理此次分配的控制组（cgroup）。

type 字段指示当前 swap 槽位所代表的映射类型：

• VSWAP_SWAPFILE： 虚拟槽位映射到 swap 设备上的物理槽位（由 slot 字段标识）

• VSWAP_ZERO： 代表一个全零页面，无需写入任何地方

• VSWAP_ZSWAP： 指向 zswap 子系统中的一个槽位（由 zswap_entry 指向）

• VSWAP_FOLIO： 对应一个当前仍驻留于 RAM 的页面（由 swap_cache 指示）

4.2.1 设计优势

这一设计的主要好处在于：页面可以轻松地在不同 swap 设备之间迁移。例如，一个 zswap 页面被推送到存储设备时，只需修改 swp_desc 结构体中的两个字段即可；该存储设备上的槽位无需在决策之前提前分配，如果某页面始终未被推出，就完全不需要在存储设备上占用任何空间。移除 swap 设备时，也无需扫描页表，因为虚拟 swap 槽位号保持不变。

4.2.2 代价与争议

当然，这一设计也有其代价：内存占用增加数倍、复杂度上升。swap table 中每个 swap entry 占用 64 位（8 字节），而 swp_desc 结构体将其扩大至最多四倍（32字节）。Pham 指出，新增的内存开销应该比表面看起来要小，因为该结构体整合了当前内核中分散存储在其他地方的信息。尽管如此，对于一个以释放内存为己任的子系统而言，这仍然是相当显著的内存消耗增加。此外，该代码在多项基准测试中出现了性能回退，尽管相较之前的版本已有较大改善。

目前，尽管这项工作有着可见的价值，但能否达到合并标准尚不明朗。在前几篇文章中承担了大量 swap 相关工作的 Kairui Song也表达了对内存开销以及系统在高负载下性能的担忧[18]，以及这里实质上破坏了swap off的语义。Chris Li 也同样担忧开销问题[19]，并指出该设计过于注重改进 zswap，而忽略了其他方面。

4.2.3 swap层级（swap tiers）

除了上述工作外，来自 Youngjun Park 的swap tiers patch set[20] 也提供了全新的想法。该系列允许将多个 swap 设备配置为分层（tier）结构：高性能设备归入一个 tier，较慢的设备归入另一个 tier。并提供了cgroup 钩子，允许管理员控制特定进程组可使用的 tier，从而为延迟敏感（或高优先级）的工作负载提供对快速 swap 设备的独占访问。

Park 的设想中，分层swap应该成为基础设施通用机制。页面应该能在不同 tier 之间的迁移，冷数据可被逐步推向更慢的存储介质。因此，假设两组补丁都能持续推进，未来两者以某种方式合并并不令人意外。

五、后续展望

除了对 Swap 子系统的深耕和推进以外，TencentOS 团队对整个内存管理领域的探索也在继续深入。今年三月份，Kairui Song 为 Linux 内核又提交了一组补丁，在社区推动了一系列讨论，为沉寂了一段时间的的 MGLRU 特性带来了众多进展。

此次补丁共八个，聚焦于 MGLRU 页面回收循环与脏页处理逻辑的优化。对于文件页面较多和涉及回写的工作负载，在测试中可有有约 30% 的性能提升，在 HDD 等慢速设备上部分工作负载提升幅度甚至超过 100%。生产环境 OOM 问题也明显减少，代码复杂度也随之降低。

值得一提的是，部分问题正是 Kairui Song 所在团队于腾讯生产环境中发现并着手解决的，体现了大规模真实场景对内核优化的重要驱动价值。同时这一系列补丁也是后续对整个内存 LRU 进行系统性优化的铺垫。目前相关补丁已提交至 Linux 内核邮件列表，等待社区审阅合入。也期待有着更精彩的后续社区发展。

注释：

1. https://lwn.net/Articles/1016136/

2. https://lwn.net/ml/all/20250916160100.31545-1-ryncsn@gmail.com/

3. https://elixir.bootlin.com/linux/v6.17.13/source/include/linux/swap.h#L294

4. https://elixir.bootlin.com/linux/v6.17.13/source/include/linux/mm_types.h#L285

5. https://elixir.bootlin.com/linux/v6.17.13/source/include/linux/swapops.h#L83

6. https://elixir.bootlin.com/linux/v6.18.6/source/arch/x86/include/asm/pgtable_64.#L183

7. https://elixir.bootlin.com/linux/v6.17.13/source/mm/swap_state.c#L39

8. https://elixir.bootlin.com/linux/v6.17.13/source/include/linux/fs.h#L485

9. https://lwn.net/Articles/495543/

10. https://docs.kernel.org/core-api/xarray.html

11. https://elixir.bootlin.com/linux/v6.17.13/source/include/linux/swap.h#L238

12. https://elixir.bootlin.com/linux/v6.19-rc5/source/mm/swap.h#L21

13. https://elixir.bootlin.com/linux/v6.19-rc5/source/mm/swap.h#L201

14. https://elixir.bootlin.com/linux/v6.19-rc5/source/mm/swap_state.c#L465

15. https://lore.kernel.org/linux-mm/20260320192735.748051-1-nphamcs@gmail.com/

16. https://lore.kernel.org/linux-mm/20260220-swap-table-p4-v1-0-104795d19815@tencent.com/

17. https://docs.kernel.org/admin-guide/mm/zswap.html

18. https://lore.kernel.org/all/CAMgjq7AQNGK-a=AOgvn4-V+zGO21QMbMTVbrYSW_R2oDSLoC+A@mail.gmail.com/

19. https://lore.kernel.org/all/CACePvbVvzh8PcF47hz+MfFu3tta5vh3oD+WpGxEL_-NrzYZG3Q@mail.gmail.com/

20. https://lwn.net/ml/all/20260217000950.4015880-1-youngjun.park@lge.com/

附录：

1. Modernizing swapping: introducing the swap table [LWN.net]https://lwn.net/Articles/1056405/

2. Modernizing swapping: the end of the swap map [LWN.net]https://lwn.net/Articles/1057102/

3. Modernizing swapping: virtual swap spaces [LWN.net]https://lwn.net/Articles/1059201/

4. https://lore.kernel.org/linux-mm/20260220-swap-table-p4-v1-0-104795d19815@tencent.com





阅读 1.0万

留言

写留言

还没有留言，来写首评